

Jose Eduardo Carrillo Peniche

Backend & Cloud Engineer — Distributed Pipelines, AWS, Industrial IoT & SCADA

Mérida, MX — remote | Dual U.S.–Mexican citizen | +52 985 852 3908
jecarrillopeniche@hotmail.es | linkedin.com/in/eillou | github.com/eillou

PROFILE

Backend engineer who ships production systems end to end — from AWS architecture down to the 4–20 mA transmitter on the tank. Over 20 months I delivered **221 features and 157 fixes across 7 revenue-generating systems**, re-architected a document pipeline from Celery to **AWS Step Functions** (1,500 PDFs in ~40 min), and built a volumetric correction engine **validated on-site against physical tank measurement**. I also own the industrial edge: Modbus acquisition, a **FUXA SCADA HMI generated from code**, and store-and-forward telemetry into AWS IoT Core. Mechatronics background means I debug the whole stack — sensor signal to API contract to compliance PDF — and I've owned a product line solo.

221 FEATURES SHIPPED	157 BUGS & VULNS FIXED	292 PRS MERGED	7 PRODUCTION CLIENTS	0 CHURN ON ROLLOUT
--------------------------------	----------------------------------	--------------------------	--------------------------------	------------------------------

EXPERIENCE

Backend & Cloud Engineer · Aivara

2024 — Present

Sole or lead engineer on two product lines: SAT tax-compliance document automation (Dictalis) and industrial IoT / SCADA telemetry across seven industrial clients in production.

DICTALIS — DOCUMENT AUTOMATION & INTEGRATIONS

- **Re-architected the OCR pipeline from Celery to AWS Step Functions** — per-document Map fan-out on Fargate with per-item fault tolerance, so one bad PDF no longer kills a 1,500-document batch. Infrastructure as code via AWS SAM/CloudFormation. Throughput: **1,500 PDFs + purchase ledger reconciled end-to-end in ~40 minutes**.
 - › Replaced the Celery watchdog with a token + heartbeat reclaim mechanism for hung executions.
 - › Built COATool: auto-detects header rows and maps columns by synonym — no manual coercion.
 - › Root-caused a double-encoding bug at the OCR source that silently returned empty tables.
- **Made the document-reading AI swappable behind one interface** — a single OCR_BACKEND switch selects Aivara's in-house OCR API or **AWS Textract**, with Textract as the fallback path for documents the primary engine can't resolve. Adding or replacing an engine is a config change, not a rewrite.
- **Shipped SAT-compliant official document generation** — replaced an ad-hoc renderer with a controlled DOCX→PDF engine (LibreOffice headless in the worker image) producing the exact layout the SAT Annex requires, with digital seal block and gap-free official folio numbering. Delivered documents pass compliance **with zero manual editing**.
- **Made critical flows idempotent and observable** — worker idempotency so a crash can't double-submit to the SAT; CloudWatch alarms on state-machine failure and notification errors.

INDUSTRIAL IOT & SCADA

- **Built the volumetric correction engine (CTL / CPL / CCF) and validated it in the field** — API MPMS 11.2.4 and 21.2 applied on-read across public and admin endpoints, with multi-tank support. Spent **a week on-site at a client plant** tuning median+EWMA smoothing and intake thresholds until detected fill events matched the physical loads observed on the floor. Ported the reference engine to Go and gated the port on **full-domain equivalence — 755,820 CTL cases, 0 differences** — rather than on the 18 published standard examples, which miss the truncation class of bug that had already shipped once.
- **Built a flow computer (tertiary element) on Raspberry Pi hardware** — the full edge chain: 4–20 mA transmitters → 8-channel Modbus RTU module → acquisition service that corrects volume on site → local Mosquitto broker → forwarder to **AWS IoT Core over mutual TLS**. A SQLite outbox **survives 30 days without a link** and drains in order when it returns. Deployed on an ED-HMI3020 panel with kiosk mode, watchdog and autostart.
- **Generated the FUXA SCADA HMI from code instead of from its editor** — one idempotent script builds the whole project (MQTT device, 20 tags, 7 views) and publishes it over the REST API, so the dashboard is reviewable in version control. Views include an ISA-5.1 P&ID per tank, the full calculation chain, alarms and cloud-queue state, and they **paginate automatically to any tank count** from a single config file.

- › Every build asserts nothing overflows the 1280×800 canvas and **aborts if it does** — a kiosk has no scrollbar, so overflow is invisible and silent. Caught five layout faults that only appear past three tanks.
- › Found and patched FUXA clipping the last 46 px of every view when its header is hidden — the DOM was correct, so nothing errored; only hit-testing the canvas revealed it.
- › Wired bidirectional control (AUTO/MAN, pump, setpoint) so the panel's indicators reflect state confirmed *by the equipment*, not what the HMI believes it sent.
- **Brought KEPServerEX analog signals to the cloud** — pulled tags from the Kepware IoT Gateway over HTTP and landed them in S3 partitioned by event date (not upload date), with a containerized alternate transport publishing to AWS IoT Core over mutual TLS at QoS 1. A malformed record is reported and skipped instead of aborting the batch. Built a hardware-free simulator so the pipeline can be exercised without plant access.
- **Designed and launched the IoT Panel** — 11-page operations console over the telemetry gateway: multi-field comparison with dual shared Y-axes, a custom signal module (prepare/despike/smooth), sustained-rise intake detection, and a compliance group. Built its design-token system, killing all hardcoded styling.

ACROSS BOTH LINES

- **Ran a continuous security-hardening program** alongside feature work: continuous dependency patching, strict CSP/CORS policies, CSRF protection, object-level authorization, input validation, and alarms on unauthorized access.
- **Owned post-deploy rollout for all 7 IoT clients** — 2–3 weeks of daily validation each (~15 client-weeks), acting as single technical point of contact through the stabilization window. **Zero clients lost to a failed rollout**; edge cases found here fed straight back into the backlog as early fixes instead of late incidents.

Process Engineer Intern · Empaques Nova

2022 — 2023

- Analyzed production data to identify efficiency improvements; documented procedures and reported results to stakeholders, strengthening standardization and traceability on the line.

TECHNICAL SKILLS

Languages	Python (primary), SQL, JavaScript, Go (porting & production ports), Bash
Backend	FastAPI, REST & SOAP integrations, async workers, Celery, Alembic migrations, idempotency & retry design
AWS	Step Functions, Lambda, Fargate/ECS, S3 (presigned flows), IoT Core, Textract, CloudWatch alarms & metric filters, SAM / CloudFormation (IaC)
Industrial	Modbus RTU/TCP, 4–20 mA instrumentation, KEPServerEX / Kepware IoT Gateway, FUXA SCADA & HMI design, ISA-5.1 P&ID, API MPMS 11.2.4 / 21.2, Raspberry Pi edge deployment, store-and-forward queues
Data & Signal	OCR pipelines (Textract), telemetry ingestion (MQTT / Mosquitto / AWS IoT Core, mTLS, QoS 1, LWT), EWMA / median filtering, despiking, event detection, data-integrity validation
Security	Dependency patching, CSP/CORS hardening, CSRF, auth guards, input allowlisting, mutual TLS, attack-surface reduction
Practices	Conventional commits, PR review, E2E/QA automation, Docker & Compose, feature flags, runbooks, on-site client validation

EDUCATION

B.S. Mechatronics Engineering

Universidad Modelo, Mérida — 2020–2024

Specialization, Cloud Software Development

Tecnológico de Software — 2024–2025

ALSO WORTH KNOWING

- Fully bilingual (Spanish / English); authorized to work in both the U.S. and Mexico with no sponsorship needed.
- Will get on a plane — field validation with the client is where I close the loop between the code and reality.